Files and exceptions

1. Introduction

So far when we have been dealing with data we have read it in, and written it out, to the screen. A much more powerful way to do things is to read and write data to a file. In this chapter we will look at how to create, read, and write to files.

There are always three steps that need to be taken when a file is used by a program:

2. Opening a file

We open a file with a command of the form

```
file_variable = open(filename,mode)
```

In this module we will look at three modes:

‘r’ open a file for reading only. The file cannot be changed or written to.

‘w’ Open a file for writing. If the file already exists, erase its contents, if it does not exist create it.

‘a’ Open a file to be written to. All data written to the file is appended to the end. If the file does not exist create it.

So for example

```
customer_file=open(‘customers.txt’, ‘r’)
```

Here the file is assumed to be in the same folder a the python file. But if it isn’t we can simply add the path:

```
customer_file=open(r‘C:\mark\com\customers.txt’,‘r’)
```

3. Writing to a file and closing it

Here we introduce the idea of a method function.

Let's say we have opened a file:

```
customer_file = open('customers.txt','w')
```

then we write to it using the write method function:

```
customer_file.write('John Cleese')
```

or

```
name1= 'John Cleese'
customer_file.write(name1)
```

We close the file using the close method function

```
Customer_file.close()
```

Example: `file_write.py`

```
# This program writes three lines of data
# to a file.
def main():
    # Open a file named philosophers.txt.
    outfile = open('philosophers.txt', 'w')

    # Write the names of three philosphers
    # to the file.
    outfile.write('John Locke\n')
    outfile.write('David Hume\n')
    outfile.write('Edmund Burke\n')

    # Close the file.
    outfile.close()

# Call the main function.
main()
```

4. Reading from a file

To read a file we can either read it all in one go using the `read` method function or read it line-by-line using the `readline` method function.

Example: file_read.py

```
# This program reads and displays the contents
# of the philosophers.txt file.
def main():
    # Open a file named philosophers.txt.
    infile = open('philosophers.txt', 'r')

    # Read the file's contents.
    file_contents = infile.read()

    # Close the file.
    infile.close()

    # Print the data that was read into
    # memory.
    print(file_contents)

# Call the main function.
main()
```

The program output is:

```
John Locke
David Hume
Edmund Burke
```

Example: `line_read.py`

```
# This program reads the contents of the
# philosophers.txt file one line at a time.
def main():
    # Open a file named philosophers.txt.
    infile = open('philosophers.txt', 'r')

    # Read three lines from the file
    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()

    # Close the file.
    infile.close()

    # Print the data that was read into
    # memory.
    print(line1)
    print(line2)
    print(line3)

# Call the main function.
main()
```

The program output is:

```
John Locke

David Hume

Edmund Burke
```

5. Adding and stripping a newline

In the example above we have ended up with an extra newline which we don't want. We can remove it using the `rstrip` method function as shown in the following code fragment:

```
infile=open('philosophers.txt','r')

line1=infile.readline()
line2=infile.readline()
line3=infile.readline()

line1=line1.rstrip('\n')
line2=line2.rstrip('\n')
line3=line3.rstrip('\n')

infile.close()
```

Frequently when we are writing data to files they are variables and so we need to add (or concatenate) a `\n` escape sequence onto the end of it before writing to a file. This is illustrated in the following code fragment:

```
name1='Fred'

people=open('friends.txt','w')

people.write(name1+'\n')

people.close()
```

6. Appending data to a file

As noted earlier if we open a file to write to it and the file already exists, then all its contents will be erased, thus we need to open existing files in the append mode. So if we wanted to add another philosopher to our data file we would write

```
outfile=open('philosophers.txt', 'a')
outfile.write('Albert Camus\n')
outfile.close()
```

7. Dealing with numerical data

To write numbers to a file we must first convert them to a string using the function `str`

```
outfile = open('numbers.txt','w')
num1=45
outfile.write(str(num1)+'\n')
outfile.close()
```

Similarly with reading from a file, we are always reading strings so we must convert them to numbers. Thus if we wanted to read our number back in from `numbers.txt` we would use:

```
infile = open('numbers.txt','r')
number=int(infile.readline())
print('The number in the file is ',number)
infile.close()
```

8. Using loops to process files

Usually files hold large amounts of data and we use a loop to process data.

Example write_sales.py

```
# This program prompts the user for sales amounts
# and writes those amounts to the sales.txt file.

def main():
    # Get the number of days.
    num_days = int(input('For how many days do ' + \
                         'you have sales? '))

    # Open a new file named sales.txt.
    sales_file = open('sales.txt', 'w')

    # Get the amount of sales for each day and write
    # it to the file.
    for count in range(1, num_days + 1):
        # Get the sales for a day.
        sales = float(input('Enter the sales for day #' + \
                            str(count) + ': '))

        # Write the sales amount to the file.
        sales_file.write(str(sales) + '\n')

    # Close the file.
    sales_file.close()
    print('Data written to sales.txt.')

# Call the main function.
main()
```

Python allows us to read the lines in a file and then automatically stops when it reached the end. The syntax is

```
for variable in file_object:
    statement
    statement
```

Example read_sales2.py

```
# This program uses the for loop to read
# all of the values in the sales.txt file
# and returns their sum as the total sales

def main():
    # Open the sales.txt file for reading.
    sales_file = open('sales.txt', 'r')

    # initialise total sales to zero
    total = 0.0

    # Read all the lines from the file.
    for line in sales_file:
        # Convert line to a float and add to total
        total = total + float(line)

    # Format and display the amount.
    print('Total sales: £',format(total, '.2f'),sep='')

    # Close the file.
    sales_file.close()

# Call the main function.
main()
```

9. Processing Records

Data stored in a file is frequently stored in **records**. A record is a complete set of data for and item, and a **field** is an individual piece of data within a record.

For example we might have a file containing data on staff in a company. Each record concerns an single member of staff and the fields correspond to name, age etc.

Example add_coffee_records.py

```
# This program adds coffee inventory records to
# the coffee.txt file.

def main():
    # Create a variable to control the loop.
    another = 'y'

    # Open the coffee.txt file in append mode.
    coffee_file = open('coffee.txt', 'a')

    # Add records to the file.
    while another == 'y' or another == 'Y':
        # Get the coffee record data.
        print('Enter the following coffee data:')
        descr = input('Description: ')
        qty = int(input('Quantity (in kg): '))

        # Append the data to the file.
        coffee_file.write(descr + '\n')
        coffee_file.write(str(qty) + '\n')

        # Determine whether the user wants to add
        # another record to the file.
        print('Do you want to add another record?')
        another = input('Y = yes, anything else = no: ')

    # Close the file.
    coffee_file.close()
    print('Data appended to coffee.txt.')

# Call the main function.
main()
```

As noted earlier we can use a `for` loop in python which will automatically move line-by-line through a file and stop at the last line.
Sometimes it is more useful to use a `while` loop which continues reading lines (usually in groups that make up individual records) until an empty line is detected.

Example search_coffee_record.py

```
# This program allows the user to search the
# coffee.txt file for records matching a
# description.

def main():
    # Create a bool variable to use as a flag.
    found = False

    # Get the search value.
    search = input('Enter a description to search for: ')

    # Open the coffee.txt file.
    coffee_file = open('coffee.txt', 'r')

    # Read the first record's description field.
    descr = coffee_file.readline()

    # Read the rest of the file.
    while descr != '':
        # Read the quantity field.
        qty = float(coffee_file.readline())

        # Strip the \n from the description.
        descr = descr.rstrip('\n')

        # Determine whether this record matches
        # the search value.
        if descr == search:
            # Display the record.
            print('Description:', descr)
            print('Quantity:', qty)
            print()
            # Set the found flag to True.
            found = True

        # Read the next description.
        descr = coffee_file.readline()

    # Close the file.
    coffee_file.close()

    # If the search value was not found in the file
    # display a message.
    if not found:
        print('That item was not found in the file.')

# Call the main function.
main()
```

10. Modifying a Record

To modify a record we

To enable us to do this we import the `os` library at the beginning of our code and then use the functions `os.remove` to delete the original file and `os.rename` to rename the temporary file.

Example modify_coffee_records.py

```
# This program allows the user to modify the quantity
# in a record in the coffee.txt file.

import os  # Needed for the remove and rename functions

def main():
    # Create a bool variable to use as a flag.
    found = False

    # Get the search value and the new quantity.
    search = input('Enter a description to search for: ')
    new_qty = int(input('Enter the new quantity: '))

    # Open the original coffee.txt file.
    coffee_file = open('coffee.txt', 'r')

    # Open the temporary file.
    temp_file = open('temp.txt', 'w')

    # Read the first record's description field.
    descr = coffee_file.readline()

    # Read the rest of the file.
    while descr != '':
        # Read the quantity field.
        qty = float(coffee_file.readline())

        # Strip the \n from the description.
        descr = descr.rstrip('\n')

        # Write either this record to the temporary file,
        # or the new record if this is the one that is
        # to be modified.
        if descr == search:
            # Write the modified record to the temp file.
            temp_file.write(descr + '\n')
            temp_file.write(str(new_qty) + '\n')

            # Set the found flag to True.
            found = True
        else:
            # Write the original record to the temp file.
            temp_file.write(descr + '\n')
            temp_file.write(str(qty) + '\n')

        # Read the next description.
        descr = coffee_file.readline()

    # Close the coffee file and the temporary file.
    coffee_file.close()
    temp_file.close()

    # Delete the original coffee.txt file.
    os.remove('coffee.txt')

    # Rename the temporary file.
    os.rename('temp.txt', 'coffee.txt')

    # If the search value was not found in the file
    # display a message.
    if found:
        print('The file has been updated.')
    else:
        print('That item was not found in the file.')

# Call the main function.
main()
```

11. Exceptions

An exception will usually lead to a red error message called a **traceback** which gives information on the error, assigning it an error type and the line where it occurred. Many exceptions can be avoided by careful programming. Python allows us white code which deals with exceptions when they occur without crashing the code.
Such code is called an **exception handler** and is written with a `try/except` statement, with the following structure:

```
try:
     Statement
     Statement
     ….
except ExceptionName:

     statement
     statement
     …
```

The `ExceptionName` is a particular error type which would be reported back if we did not use exception handling. Thus if we tried to enter ‘four’ when 4 was expected we would get a `ValueError.` Or if we tried to open a file to read which didn’t exist we would get a `IOError.` We could catch these specific types of error by listing them as the `ExceptionName.`

The inclusion of an `ExceptionName` is optional. If none is included the code enters the second set of statements if *any* exception occurs.

Example: gross_pay2.py

```
# This program calculates gross pay.

def main():
    try:
        # Get the number of hours worked.
        hours = int(input('How many hours did you work? '))

        # Get the hourly pay rate.
        pay_rate = float(input('Enter your hourly pay rate: '))

        # Calculate the gross pay.
        gross_pay = hours * pay_rate

        # Display the gross pay.
        print('Gross pay: $', format(gross_pay, ',.2f'), sep='')
    except ValueError:
        print('ERROR: Hours worked and hourly pay rate must')
        print('be valid numerical data.')

# Call the main function.
main()
```

Supportive Reading

Tony Gaddis, *Starting Out with Python*, 5/e Pearson, 2022, Chapter 6.

Note that more material on exception handling can be found on pages 363-376 of Gaddis.